CityStreet: Multi-View Crowd Counting Dataset

Qi Zhang, Antoni B Chan.
Department of Computer Science, City University of Hong Kong

Aug 12, 2019
Copyright (c) 2019, City University of Hong Kong

This is the explanation document for the multi-view crowd counting datasets used in the “wide-area crowd counting” paper, including two existing datasets PETS2009 and DukeMTMC, and our proposed dataset CityStreet.

1. CityStreet:

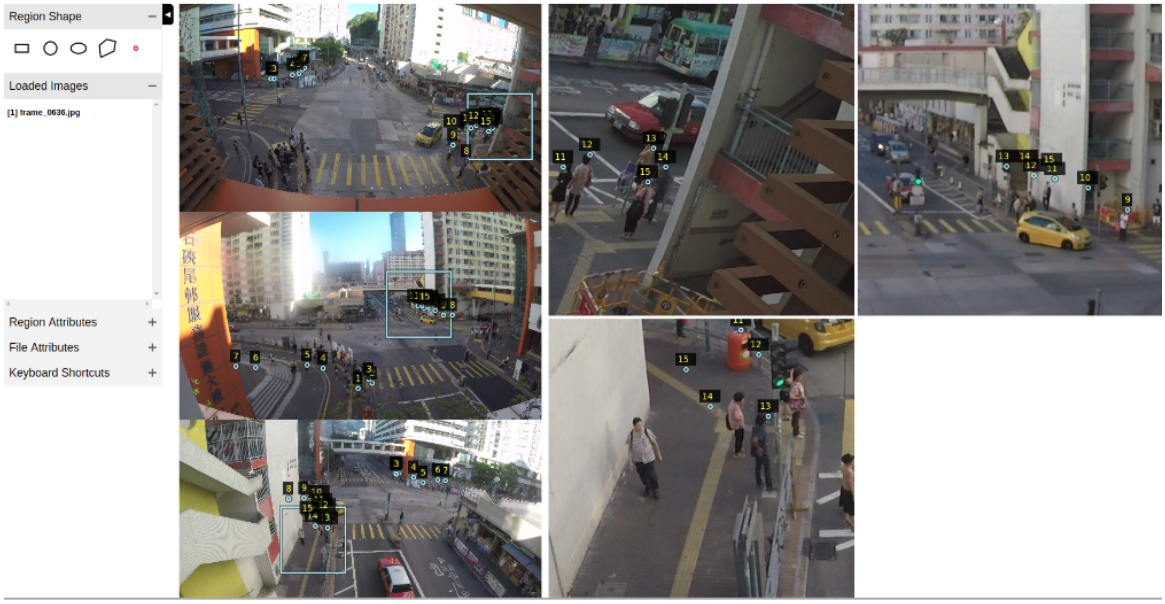
Image frames:

The City Street dataset was collected on the street of Hong Kong. The current version includes 3 cameras, camera1, 3 and 4 (denoted as view 1, 2 and 3), and their locations are shown on the figure. The image frame resolution is 2704*1520.



Training: frame_0636.jpg---frame_1234.jpg, 300 images in total.
Testing: frame_1236.jpg---frame_1634.jpg, 200 images in total.

Labeling_tool:



The multi-view labeling tool instruction can be found in the PETS2009/labeling_tool folder. But the labeling tool for City Street is a little different due to the long distance of the people to the camera. The left 3 canvases are used to show the labels or select regions to annotate. The right 3 canvases are used for annotating, which is similar to the PETS labeling tool.

Labels:
"via_region_data_view1.json", "via_region_data_view2.json" and "via_region_data_view3.json" are the camera view labels (x, y) of all people in the 500 images (h*w = 1520*2704). And if (x, y) is out of the ROI of the view, the label should be ignored, which means the corresponding ID doesn't exist in that view.
"GP_pmap_height.json" contains the ground-plane labels of all the people (h*w = 768*640, height = [1560, 1980] mm). The ground plane coordinates of the people are calculated from the corresponding camera view labels by searching the people height **h** and then deciding the average ground-plane location (**x**, **y**) projected from camera views.

Projection code:
The calibration toolbox used: http://www.vision.caltech.edu/bouguetj/calib_doc/htmls/parameters.html.
The projection code includes "Image2World" and "World2Image" functions, which are written in python. The input of the image2world projection is the camera view coordinate (x, y, height): 0<x<2704, 0<y<1520, and the height is calculated through searching among (1560, 1980, 10) mm.
First, the coordinate range (x, y) in the label json files is 0~2704 and 0~1520; if the coordinate (x, y) is from input image (h*w = 380*676), resize it to the original image resolution: x = x*4, y = y*4.
Then use the Image_to_World projection function to get the world coordinate wc. After we get the world coordinate wc, it should be further normalized through:

```
bbox = [352*0.8, 522*0.8]
image_size = [380, 676]
resolution_scaler = 76.25
view1_wc_offset = view1_wc / resolution_scaler
view1_wc_offset_x = view1_wc_offset[:, 0:1] + bbox[0]  # * resolution_scaler
view1_wc_offset_y = view1_wc_offset[:, 1:2] + bbox[1]  #
```

(view_wc_offset_x, view1_wc_offset_y) is the final world coordinate and should be in the range of (h=768, w=640). The world2image projection will be the reverse of the above process.

GT_density_maps:
"camera_view" folder contains the density map of each camera view. The original dmap resolution is h*w = 576*768, Gaussian kernel std = 10. But the network input dmaps are downsized to 1/2, whose resolution is h*w = 288*384.
"ground_plane" folder contains the scene-level density maps.
The dmap resolution is h*w = 710*610, Gaussian kernel std = 10.
In the network, the output is 1/4 of the dmap resolution, whose size is h*w = 177*152.

ROI_maps:
1) ROIs:
contains the camera view masks and the ground-plane mask of each view.
2) Normalization_maps:
contains the normalization weight proposed for late fusion model to handle the density sum change after the projection.
3) Weighted_maps:
The weighted_maps indicate the weights used in the comparison method 1 "dmap_weighted".
4) Distance_maps:

The distance maps are used for scale selection in the MVMS model. The distance maps are calculated through assuming all pixel height in the camera view is average height in the 3D world.

5) Scene_map:

The scene map is used for visualization. The scene map resolution is h*w = 461*438. For visualization, the scene map coordinate should be consistent with ground-plane density map (resolution h*w = 192*160).

First resize the scene map to h*w = 412*392, denoted as GP_scene_map;

Then, increase dmap's resolution by 2 times (h*w = 384*320), denoted as GT_dmap_i.

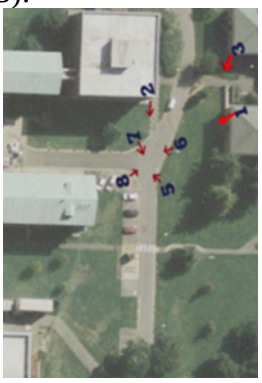
Finally, pad the dmap using following code:

```
GT_dmap_padded = np.zeros((GP_scene_map.shape[0], GP_scene_map.shape[1]))
w0 = GT_dmap_i.shape[1]
h0 = GT_dmap_i.shape[0]
dx = 36
dy = 5
GT_dmap_padded[2*dy:2*dy+h0, 2*dx:2*dx+w0] = GT_dmap_i.copy()
```

2. **PETS2009**

Image frames:

The original images can be downloaded from the original link of PETS 2009 dataset. In our paper, following sequences are used (view 1, 2, 3):



Training sequences: 1105 images in total.

S1L3 Time_14-17: frame_0000.jpg-frame_0090.jpg, 91 images

S1L3 Time_14-33: frame_0000.jpg-frame_0343.jpg, 344 images

S2L2 Time_14-55: frame_0003.jpg-frame_0432.jpg, 430 images

S2L3 Time_14-41: frame_0000.jpg-frame_0239.jpg, 240 images

Testing sequences: 794 images in total.

S1L1 Time_13-57: frame_0000.jpg-frame_0220.jpg, 221 images

S1L1 Time_13-59: frame_0000.jpg-frame_0240.jpg, 241 images

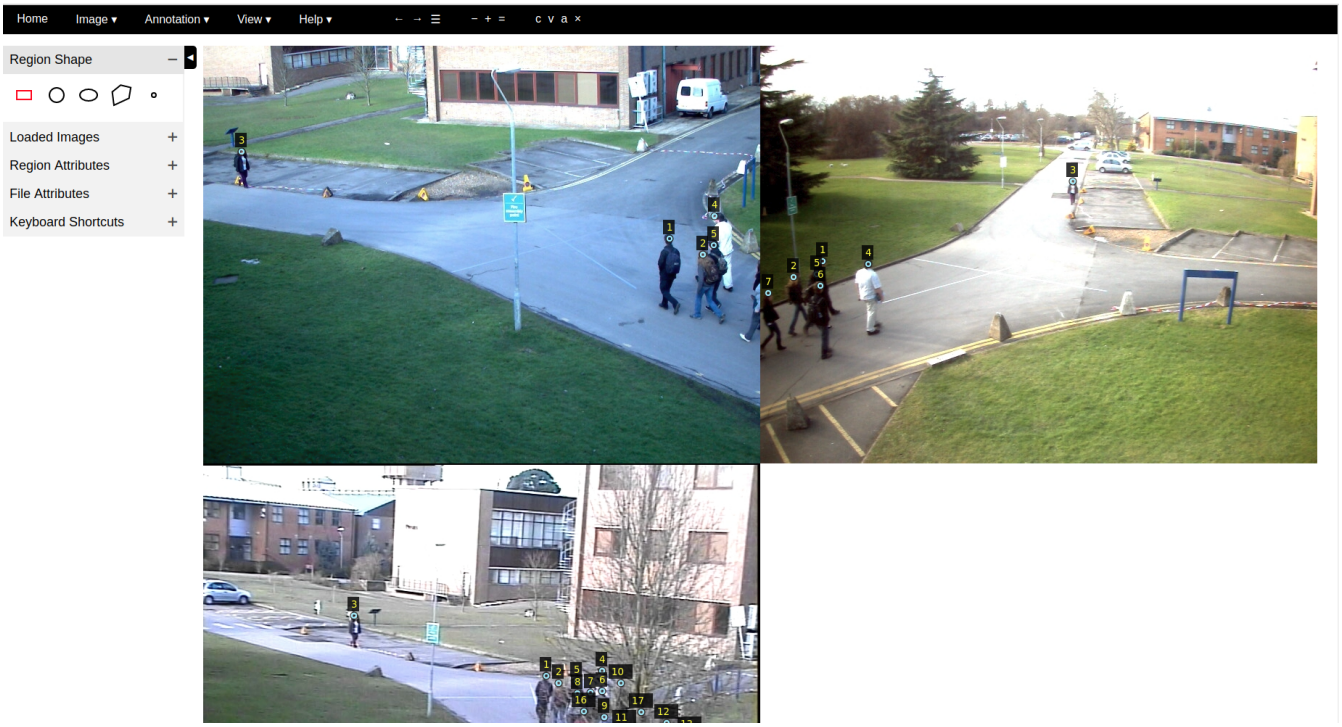
S1L2 Time_14-06: frame_0000.jpg-frame_0200.jpg, 201 images

S1L2 Time_14-31: frame_0000.jpg-frame_0130.jpg, 131 images

The resolution of the original images are h*w = 576*768. But they are downsized to 1/2 before being input into the network.

Labeling_tool:

A multi-view labeling tool based on VIA 1.0.4 (http://www.robots.ox.ac.uk/~vgg/software/via/) is provided for image annotating. The multi-view labeling tool contains two parts: the html-based interactive canvas and the "projection.js" part for coordinate projection across views. More details can be found in the "**The multi-view labeling tool instruction.pdf**" in the "labeling_tool" folder.



Labels:
To obtain the annotations across all views, we use the View 1 annotations provided by original PETS2009 authors and project them to other views followed by manual annotations to get all the people head locations in the images. The same person is assigned with the same ID across views.
"via_region_data_view1.json", "via_region_data_view2.json" and "via_region_data_view3.json" are the camera view labels (x, y) of all people (h*w = 576*768). And if x<50 and y<50, the label should be ignored (labels on the top-left corner are not valid), which means the corresponding ID doesn't exist in that view.
"GP_pmap_height.json" is the ground-plane label of the people (h*w = 710*610, height = [1560, 1980] mm). The ground plane coordinates of the people are calculated from the corresponding camera view labels by searching the people height **h** and then deciding the average ground-plane location (**x**, **y**) projected from camera views.

Projection code:
The projection code uses the camera calibration parameters and the calibration method provided by the original PETS2009 dataset. The projection code includes "Image2World" and "World2Image" functions, which are written in python.
The input of the image2world projection is the camera view coordinate (x, y, height): 0<x<768, 0<y<576, and height is calculated through searching among [1560, 1980] mm.
After we get the world coordinate wc through image2world projection, it should be further normalized through:

```
bbox = [-31, 29, -45, 25]
image_size = [576, 768]
resolution_scaler = 10
view1_wc_offset = view1_wc / 1000
view1_wc_offset_x = (view1_wc_offset[:, 0:1] - bbox[0]) * resolution_scaler
view1_wc_offset_y = (view1_wc_offset[:, 1:2] - bbox[2]) * resolution_scaler
```

(view_wc_offset_x, view1_wc_offset_y) is the final world coordinate and should be in the range of (w=0~610, h=0~710). The world2image projection will be the reverse of the above process.

GT_density_maps:
"camera_view" folder contains the density map of each camera view. The original dmap resolution is h*w = 576*768, Gaussian kernel std = 10. But the network input dmaps are downsized to 1/2, whose resolution is h*w = 288*384.
"ground_plane" folder contains the scene-level density maps.
The dmap resolution is h*w = 710*610, Gaussian kernel std = 10.
In the network, the output is 1/4 of the dmap resolution, whose size is h*w = 177*152.

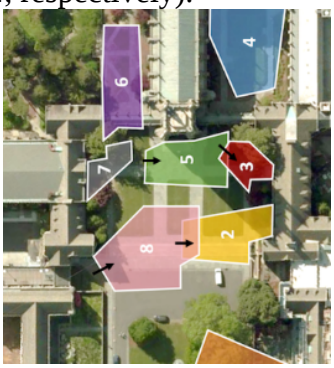
ROI_maps:
1) ROIs:
contains the camera view masks and the ground-plane mask of each view.
2) Normalization_maps:
contains the normalization weight proposed for late fusion model to handle the density sum change after the projection.
3) Weighted_maps:
The weighted_maps indicate the weights used in the comparison method 1 “dmap_weighted”.
4) Distance_maps:
The distance maps are used for scale selection in the MVMS model. The distance maps are calculated through assuming all pixels in the camera view are of average height in the 3D world.

5) Scene_map:
The scene map is used for visualization. The scene map resolution is h*w = 918*690. For visualization, the scene map coordinate should be consistent with output ground-plane density map (resolution h*w = 177*152).
First resize the scene map GP_scene_map to h*w = 510*384, denoted as GP_scene_map;
Then, flip the gp dmap using np.fliplr and increase its resolution by 2 times (h*w = 354*304), denoted as GT_dmap_i.
Finally, pad the GT_dmap:

```
GT_dmap_padded = np.zeros((GP_scene_map.shape[0], GP_scene_map.shape[1]))
w0 = GT_dmap_i.shape[1]
h0 = GT_dmap_i.shape[0]
dx = 18
dy = 15
GT_dmap_padded[2*dy:2*dy+h0, 2*dx:2*dx+w0] = GT_dmap_i.copy()
```

3. **DukeMTMC**:

Image frames:
The original images can be downloaded from the link of DukeMTMC dataset. Camera 2, 3, 5, 8 are selected (denoted as view 1, 2, 3, and 4, respectively).



We don’t provide the frames directly, which should be sampled from the original DukeMTMC videos. The frames are sampled like this:
1) Sample 2 frames in every second from each video.
This command can be used:

```
ffmpeg -i video.webm -vf fps=2 frame_%04d.jpg -hide_banner
```

And name the frames like: “frame_00001.jpg, frame_00002.jpg, ...” for each video.

2) Camera 2 starts from video 1 frame_00258.jpg to video 5 frame_01067.jpg, 5934 images in total; At the same time renaming them from frame_0001.jpg to frame_5934.jpg in order.
Camera 3 starts from video 0 frame_00749.jpg to video 5 frame_00276.jpg, 5934 images in total; Renaming them in the above way.
Camera 5 starts from video 1 frame_00378.jpg to video 5 frame_01187.jpg, 5934 images in total; Renaming them in the above way.
Camera 8 starts from video 0 frame_00098.jpg to video 4 frame_00907.jpg, 5934 images in total; Renaming them in the above way.

3) Select 1 frame every other 6 frames (frame_0001.jpg, frame_0007.jpg, ...), 989 images in total for each view.
4) The first 700 images for training, and rest 289 for testing.

Labels:
The camera view labels (DukeMTMC_camera2_head.json, DukeMTMC_camera3_head.json, DukeMTMC_camera5_head.json, DukeMTMC_camera8_head.json) are extracted from the original gt provided by the DukeMTMC dataset.
Since the ground-truth only includes the disjointed regions between camera views. The ground-truth of camera 2, 3, 5 is projected to camera view 8, as in DukeMTMC_camera8_head_proj.json

Projection code:
The projection code uses the camera calibration parameters and the calibration method provided by the original DukeMTMC dataset. The projection code includes “Image2World” and “World2Image” functions, which are written in python.
The input of the image2world projection is the camera view coordinate (x, y, height): 0<x<1920, 0<y<1080. Since the original ground-truth of DukeMTMC is disjointed and no across-view labels are available, all people height is assumed to be 1.75m. So, height = 1.75.
The coordinate range (x, y) in the jsons is 0~640 and 0~360, or if the coordinate (x, y) is the location from input image (640*360), we need to resize it to the original image resolution first:
x = x*3, y = y*3.
Then use the image2world projection function to get the world coordinate wc. After we get the world coordinate wc, it should be further normalized through:

```
ph = 1.75
bbox = [50, 50]
resolution_scaler = 4
v1_wc_coords_offset = (v1_wc_coords + bbox)*resolution_scaler
```

v1_wc_coords_offset = (view_wc_offset_x, view1_wc_offset_y) is the final world coordinate and should be in the range of (h=480, w=640).
The world2image projection will be the reverse of the above process.

GT_density_maps:
"camera_view" folder contains the density map of each camera view (train and test folder). The original dmap resolution is h*w = 360*640, Gaussian kernel std = 10.

“DukeMTMC_view4_dmap_10.h5” only includes the people in the original DukeMTMC ground-truth of camera 8. And “DukeMTMC_view4_dmap_10_proj.h5” includes labels projected from other camera views (camera2, 3, and 5) to camera 8.
"ground_plane" folder contains the scene-level density maps. The dmap resolution is h*w = 480*640, Gaussian kernel std = 10. In the network, the output is 1/4 of the dmap resolution, whose size is h*w = 120*160.

ROI_maps:
1) ROIs: include the missing label ROI of camera 8 (view 4).
2) Normalization_maps
3) Weighted_maps
4) Distance_maps
5) Scene_map:
The scene map is used for visualization. The scene map resolution is h*w = 785*726. For visualization, the scene map coordinate should be consistent with ground-plane density map pred_dmap_MVMS_i (resolution h*w = 120*160).
First resize the scene map to h*w = 240*216, denoted as GP_scene_map;

```
pred_dmap_MVMS_i = cv2.resize(pred_dmap_MVMS_i, (0, 0), fx=2, fy=2)
# pad the dmap to the size of the scene map
w = GT_dmap_i.shape[1]
h = GT_dmap_i.shape[0]
dmap_padded = np.zeros(GP_scene_map.shape[:2])
pred_dmap_MVMS_i_padded = dmap_padded.copy()
dx = 8
dy = 0
pred_dmap_MVMS_i_padded[2*dy:2*dy + h, 2*dx:2*dx + w] = pred_dmap_MVMS_i.copy()
```

If you use the dataset or the labels, please remember to cite our paper:

```
@inproceedings{zhang2019wide,
title={Wide-Area Crowd Counting via Ground-Plane Density Maps and Multi-View Fusion CNNs},
author={Zhang, Qi and Chan, Antoni B},
booktitle={Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition},
pages={8297--8306},
year={2019}
}
```

If you have questions, contact qzhang364@my.cityu.edu.hk.

Acknowledgements:

This work was supported by a grant from the Research Grants Council of the Hong Kong Special Administrative Region, China (CityU 11212518).